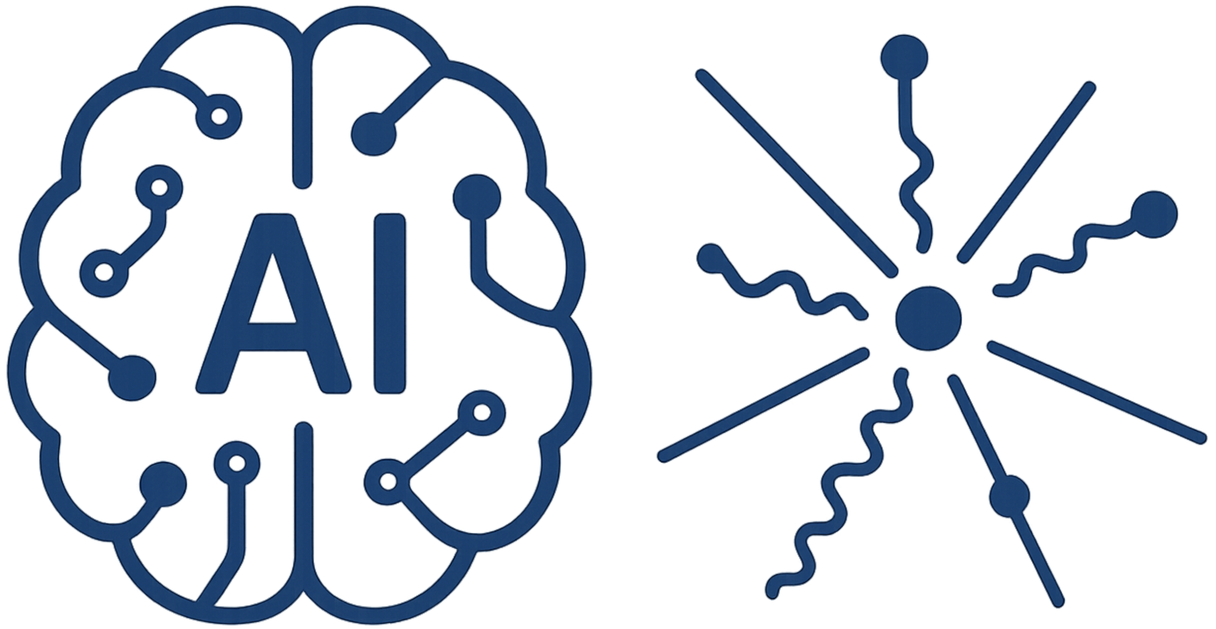
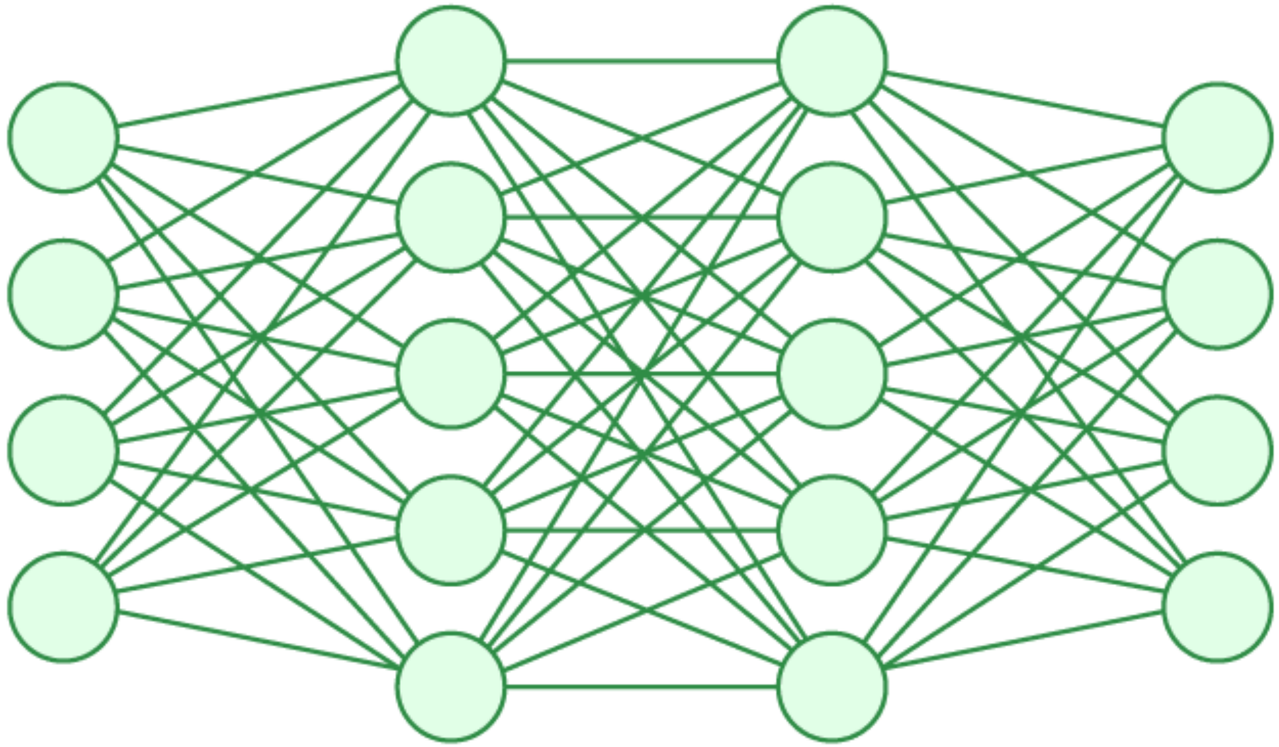
Introduction to

AI-Driven HEP

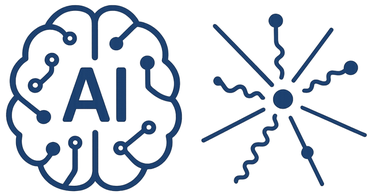
S. A. Fard - School of Physics (IPM)





Session 9

Neural Networks



Reference

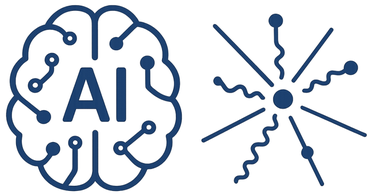
[1] James, Gareth, et al. "Statistical learning." An introduction to statistical learning: With applications in Python. (2023)

[2] Hastie, Trevor. "The elements of statistical learning: data mining, inference, and prediction." (2009)

[3] Bishop, Christopher M., Pattern recognition and machine learning. springer, (2006)

[4] Chollet, Francois. Deep learning with Python. (2021)

[5] Shalev-Shwartz, Shai, and Shai Ben-David. Understanding machine learning: From theory to algorithms. (2014)



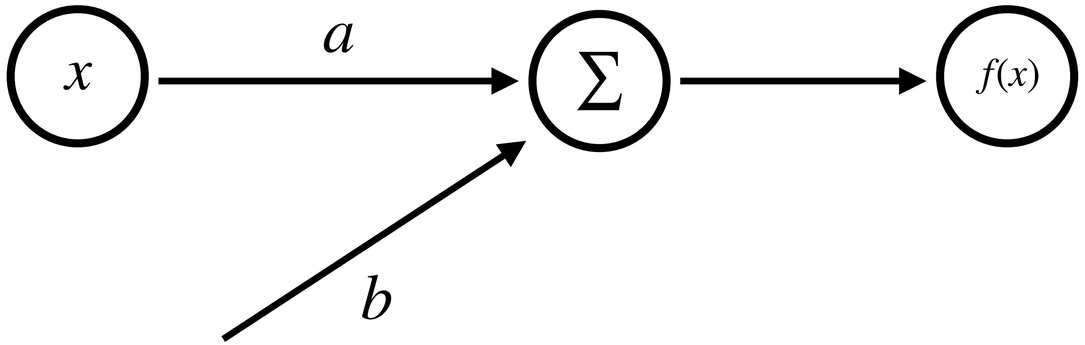
Motivation

Remember the challenges of previous session

- Finding the optimum value for parameters
- Generalizing the approach to more complicated functions

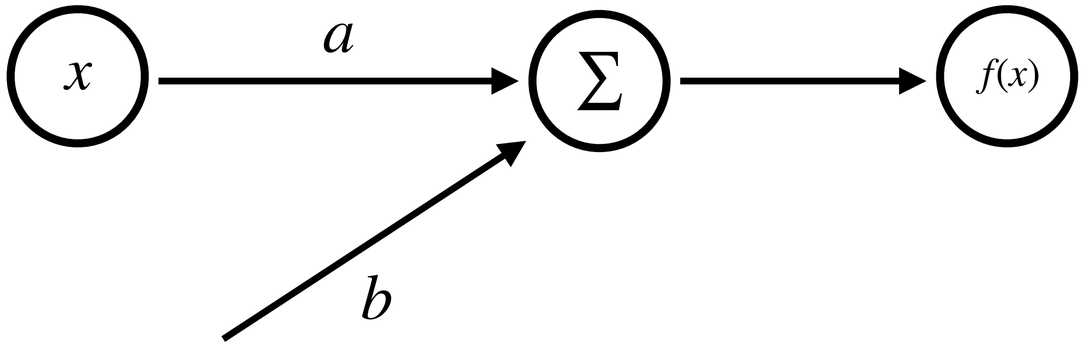
Optimization

$$y = ax + b$$



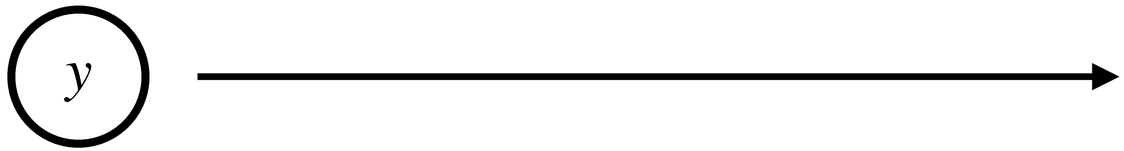
Optimization

$$y = ax + b$$



Loss Function

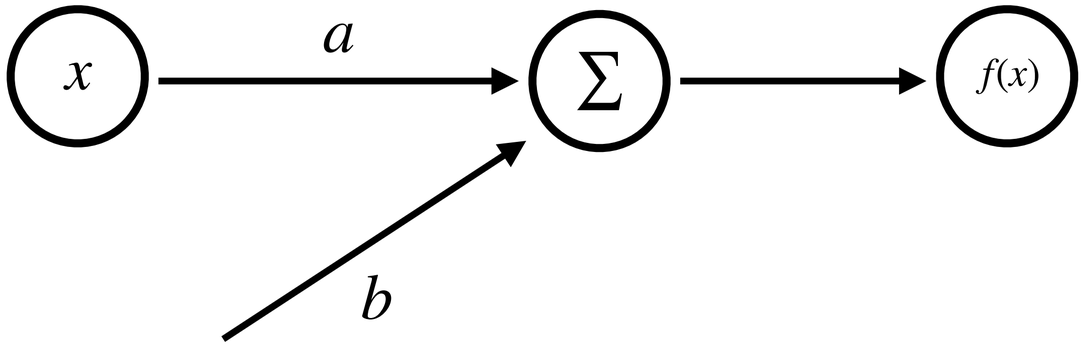
Squared-Error



$$L = \sum_i (y_i - ax_i - b)^2$$

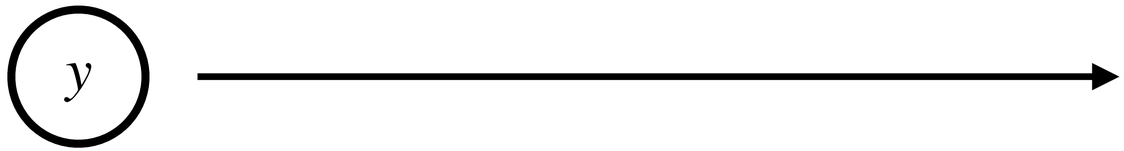
Optimization

$$y = ax + b$$



Loss Function

Squared-Error

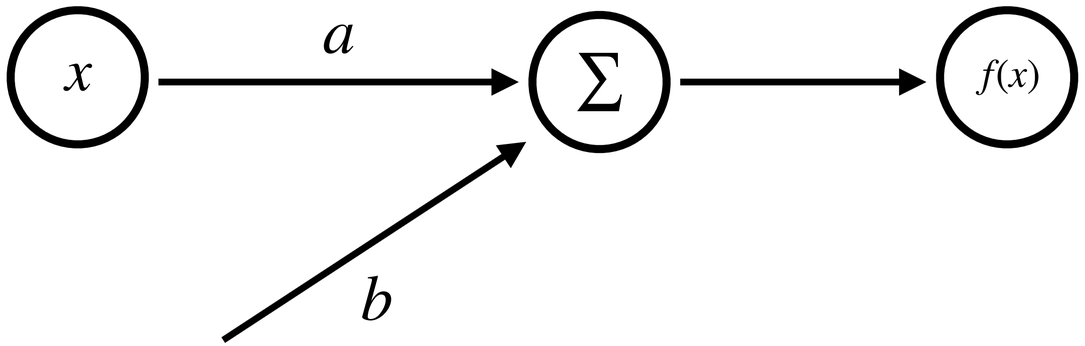


$$L = \sum_i (y_i - ax_i - b)^2$$

More ? wait

Optimization

$$y = ax + b$$



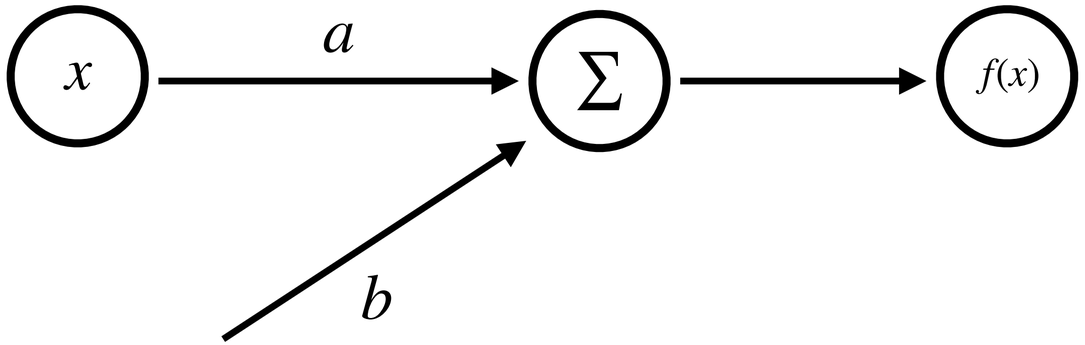
Gradient Descent

$$a_{new} = a_{old} - \frac{\partial L(a_{old}, b_{old})}{\partial a_{old}}$$

$$b_{new} = b_{old} - \frac{\partial L(a_{old}, b_{old})}{\partial b_{old}}$$

Optimization

$$y = ax + b$$



Gradient Descent

$$a_{new} = a_{old} - \frac{\partial L(a_{old}, b_{old})}{\partial a_{old}}$$

$$b_{new} = b_{old} - \frac{\partial L(a_{old}, b_{old})}{\partial b_{old}}$$

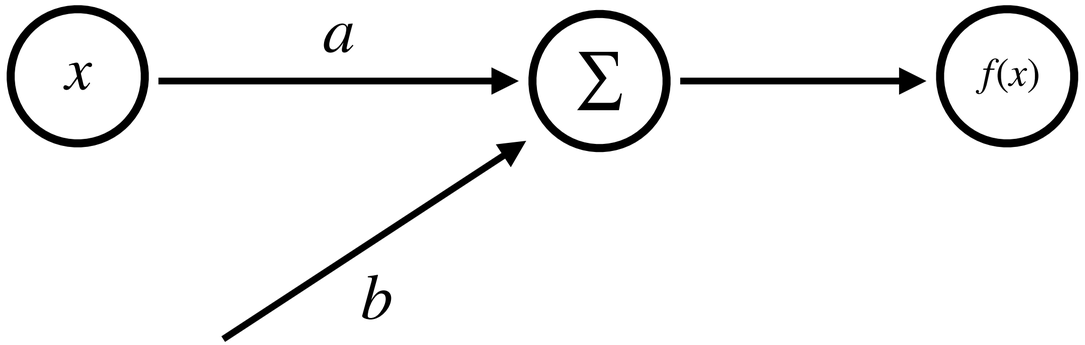
Solve for this data points

starting with (a=1,b=1)

$$x_1 = 1 \qquad y_1 = 1$$
$$x_2 = 2 \qquad y_2 = 2$$
$$x_3 = 3 \qquad y_3 = 2$$

Optimization

$$y = ax + b$$



$$a_{new} = a_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial a_{old}}$$

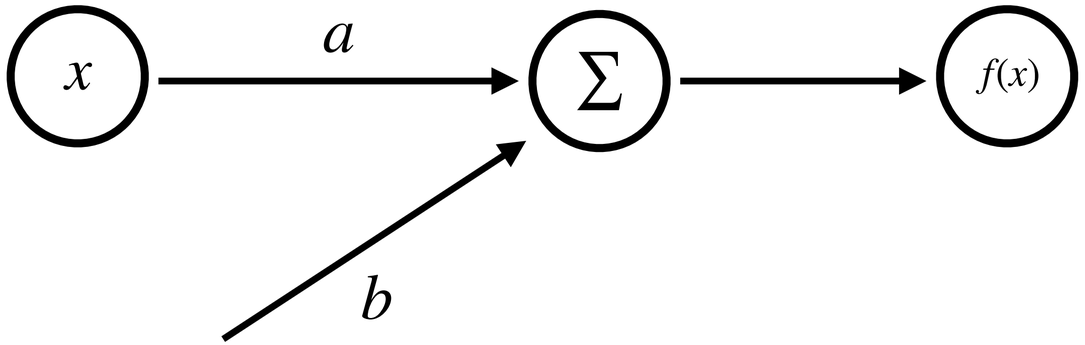
$$b_{new} = b_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial b_{old}}$$

Gradient Descent

There is a need for the learning rate

Optimization

$$y = ax + b$$



$$a_{new} = a_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial a_{old}}$$

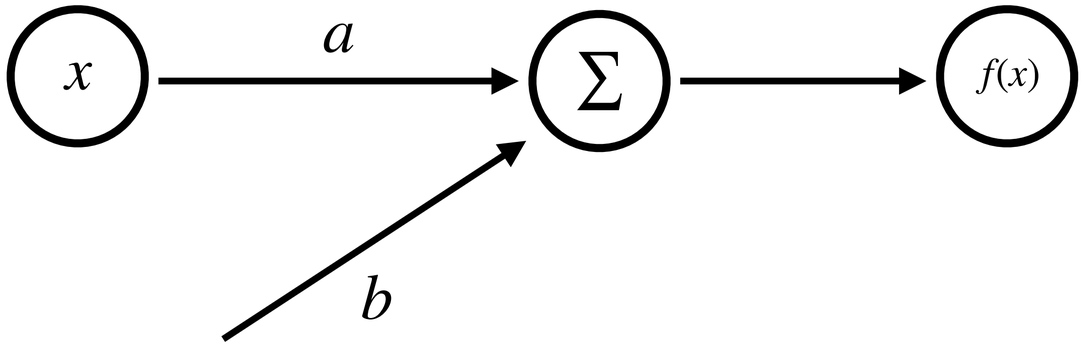
Stochastic Gradient Descent

$$b_{new} = b_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial b_{old}}$$

The whole data set or a subset of data (mini-batch)

Optimization

$$y = ax + b$$



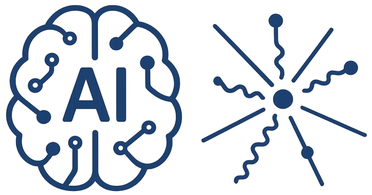
$$a_{new} = a_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial a_{old}}$$

Stochastic Gradient Descent

$$b_{new} = b_{old} - \eta \frac{\partial L(a_{old}, b_{old})}{\partial b_{old}}$$

More ?

For example see Adam Optimizer

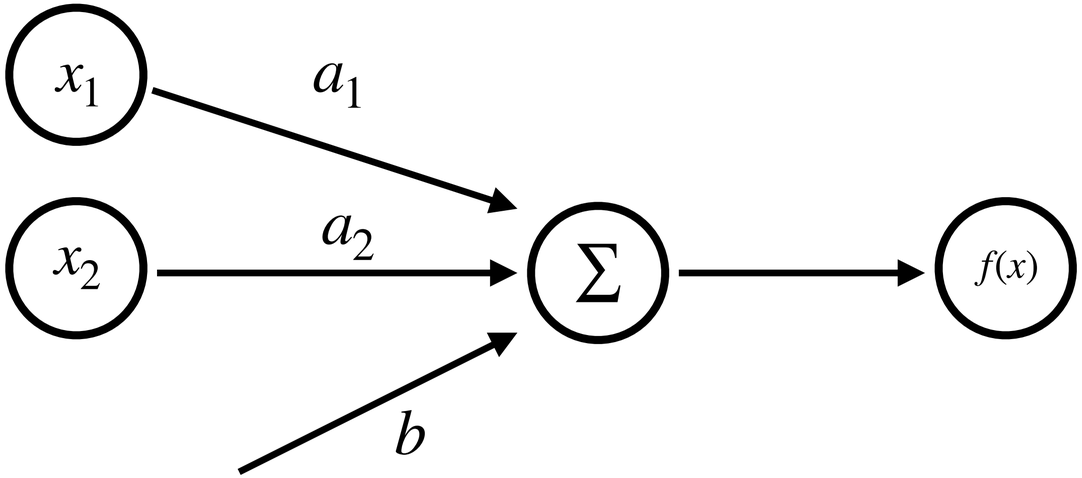


Generalization

Generalization

Adding more features

$$y = a_1 x_1 + a_2 x_2 + b$$

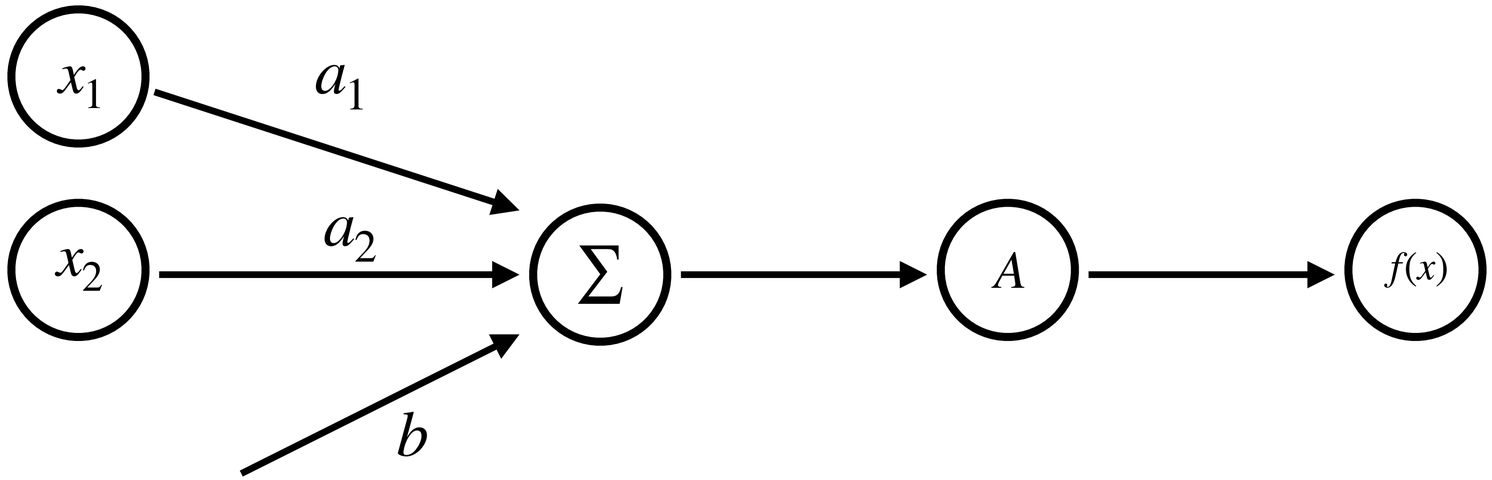


Generalization

Adding more features

Capturing Non-linearity

$$y = A(a_1 x_1 + a_2 x_2 + b)$$

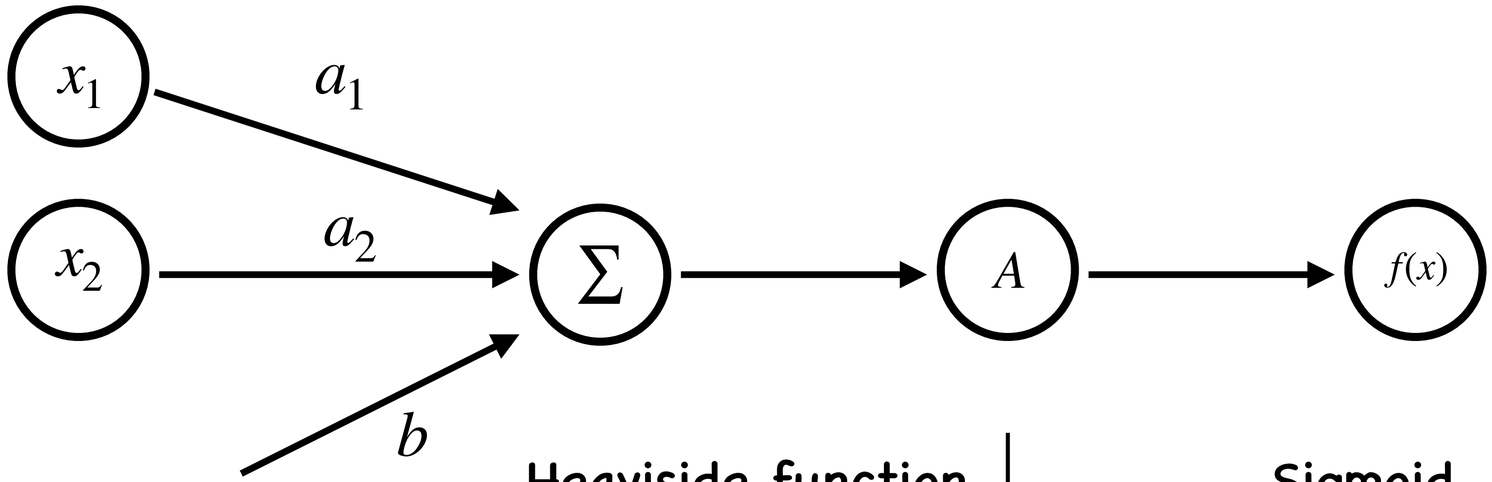


Generalization

Adding more features

Capturing Non-linearity

$$y = A(a_1 x_1 + a_2 x_2 + b)$$



Heaviside function

$$H(X) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

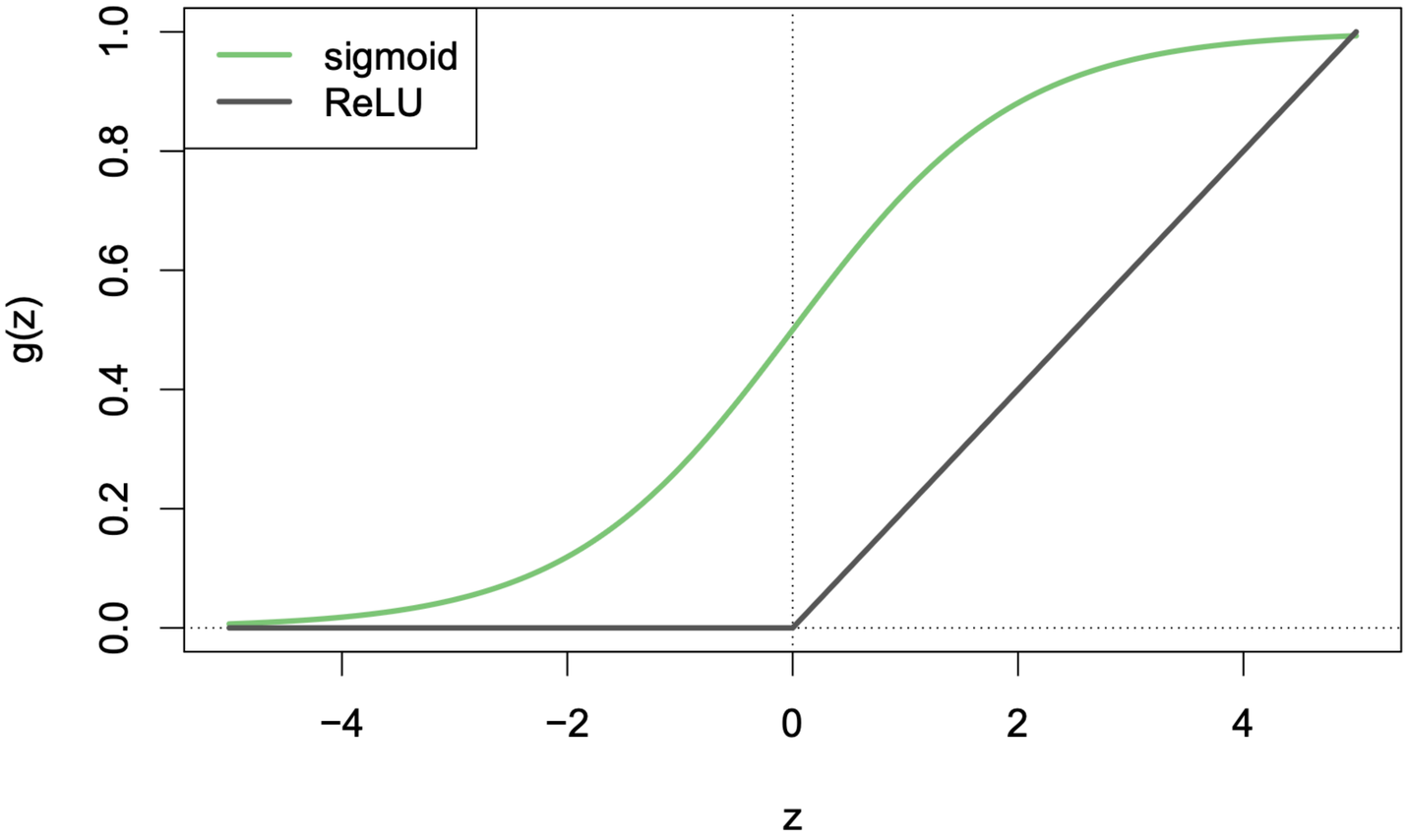
Sigmoid

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$$

Rectified Linear Unit (ReLU)

$$\sigma(x) = \max(0,x)$$

Generalization



ReLU

Simple derivative

Fast calculation

Sigmoid

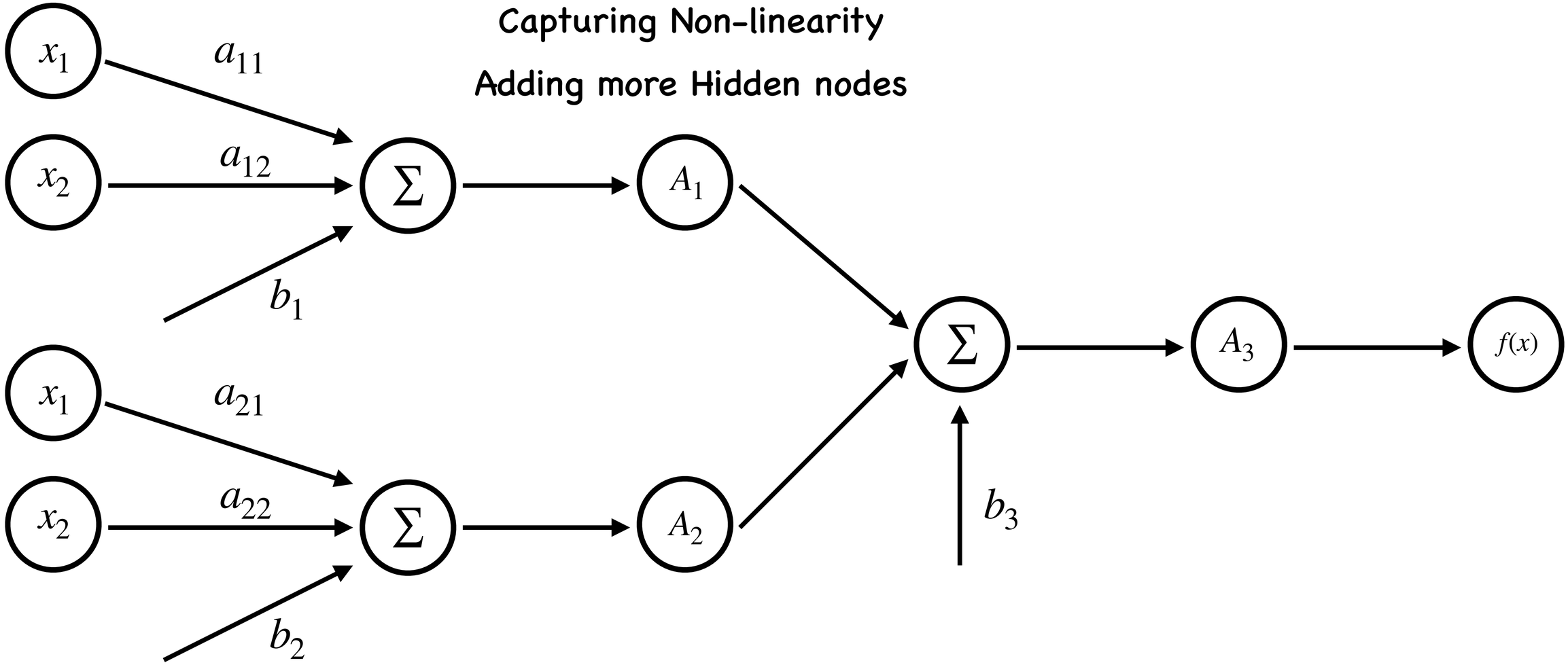
convert a linear function into probabilities between zero and one

Generalization

Adding more features

Capturing Non-linearity

Adding more Hidden nodes

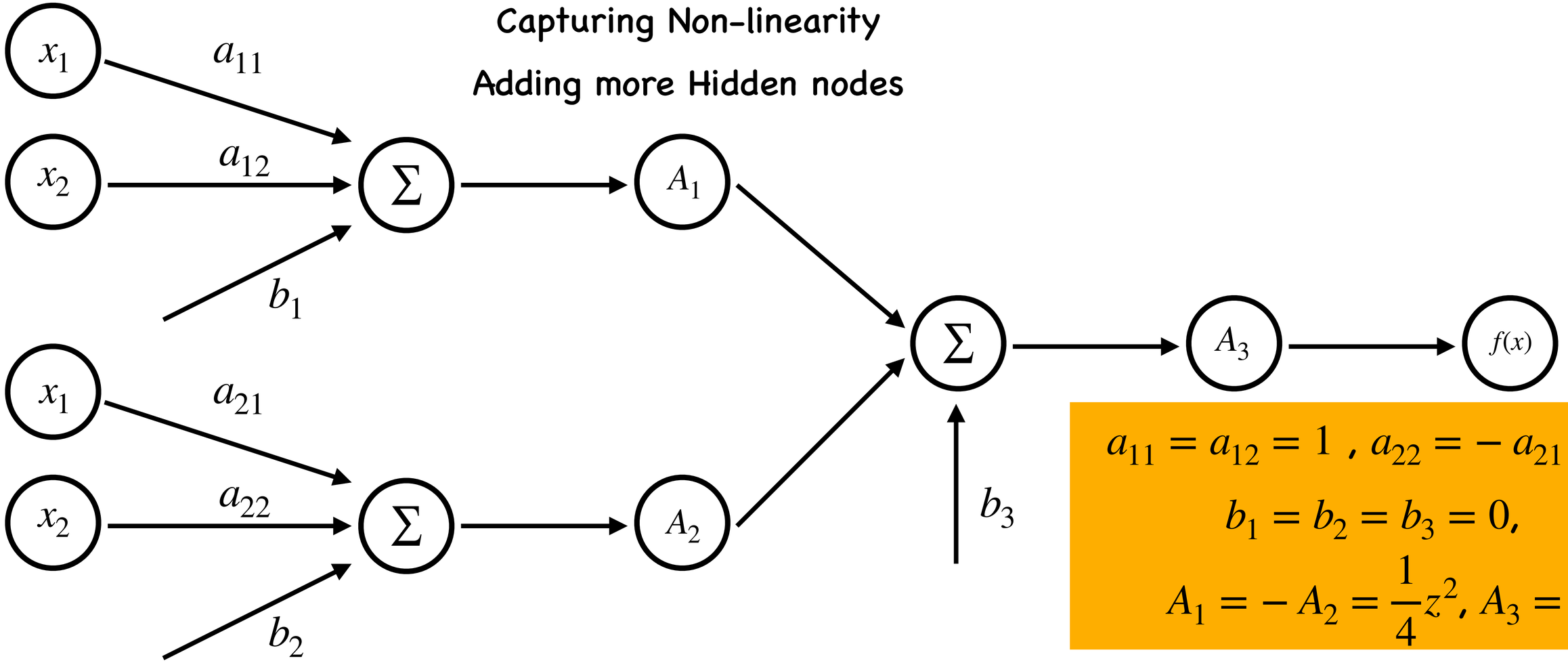


Generalization

Adding more features

Capturing Non-linearity

Adding more Hidden nodes

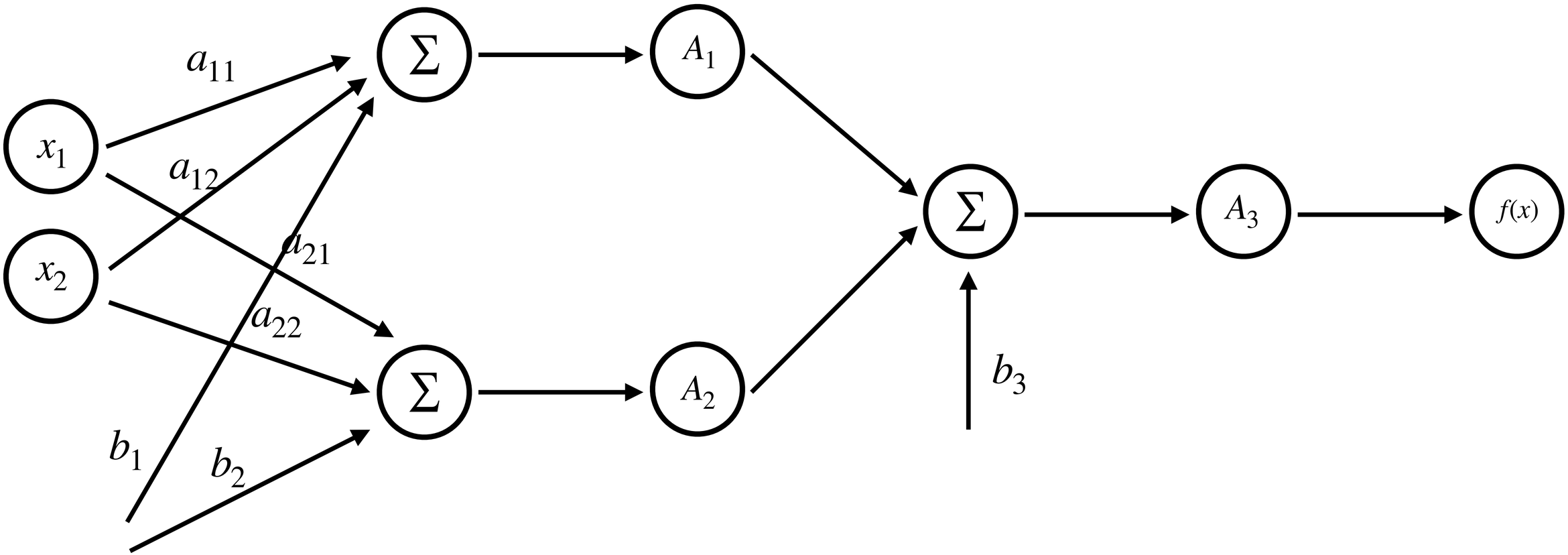


$$a_{11} = a_{12} = 1\ ,\ a_{22} = -\,a_{21} = 1$$

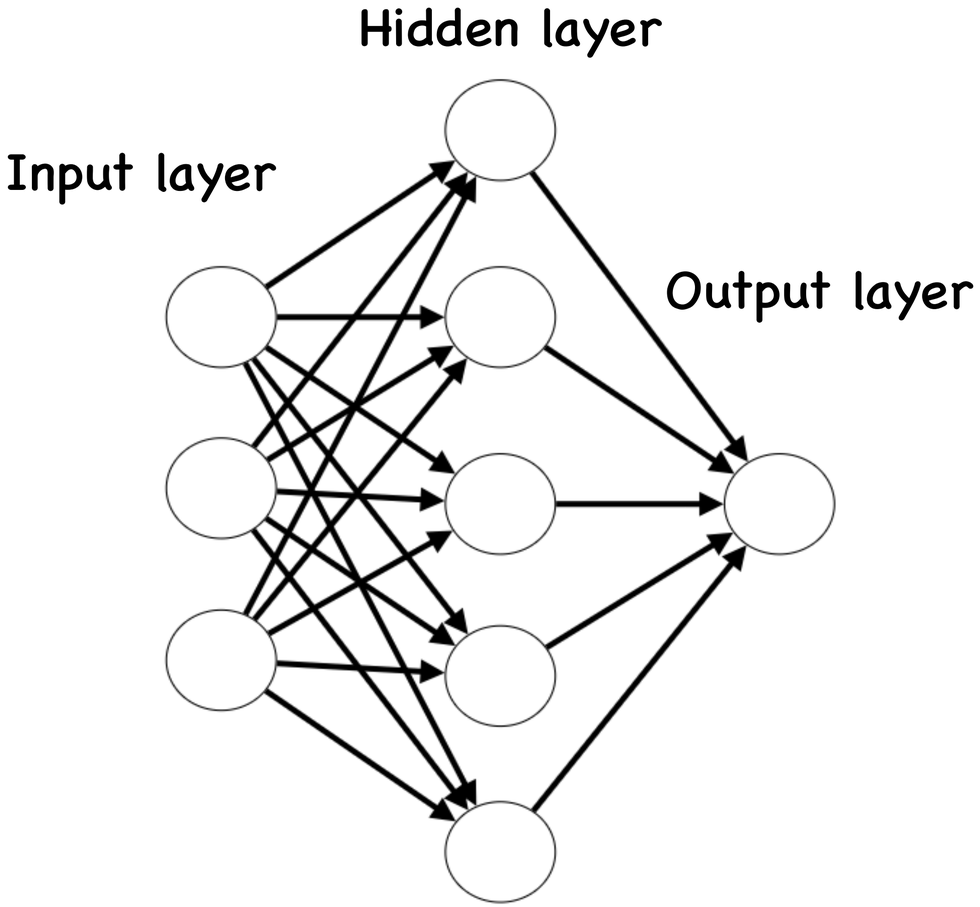
$$b_1 = b_2 = b_3 = 0,$$

$$A_1 = -\,A_2 = \frac{1}{4}z^2,\ A_3 = z$$

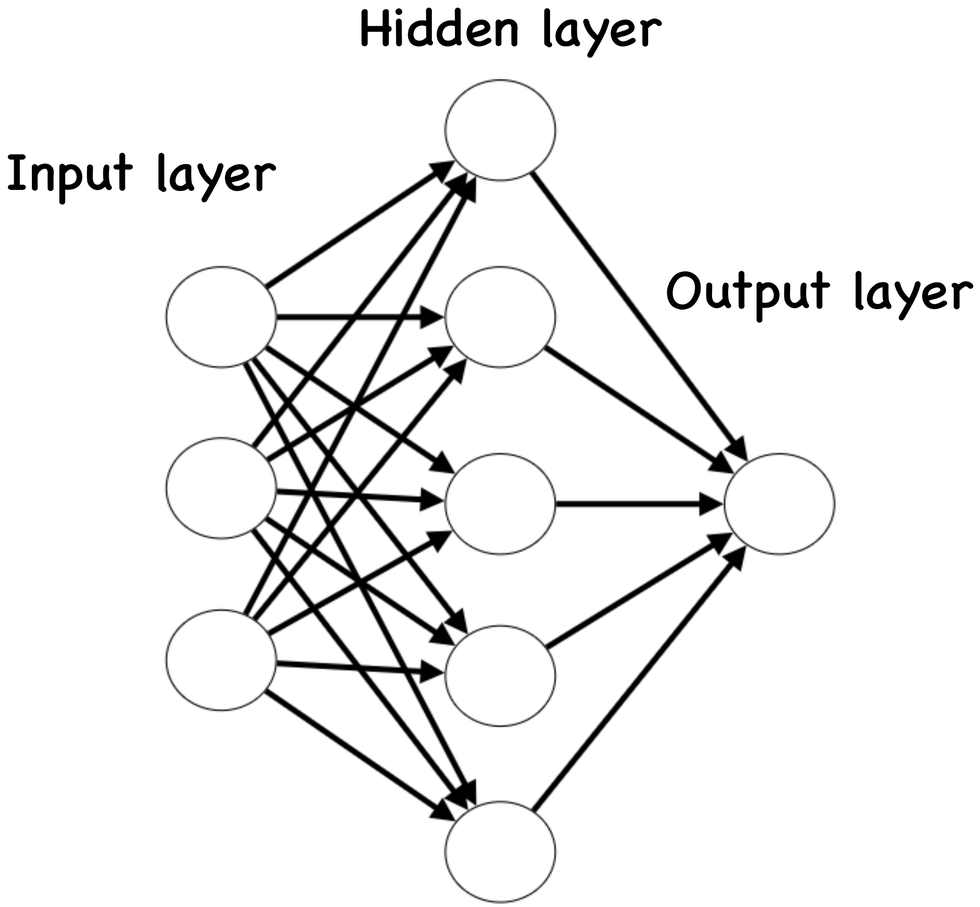
Neural Network



Neural Network



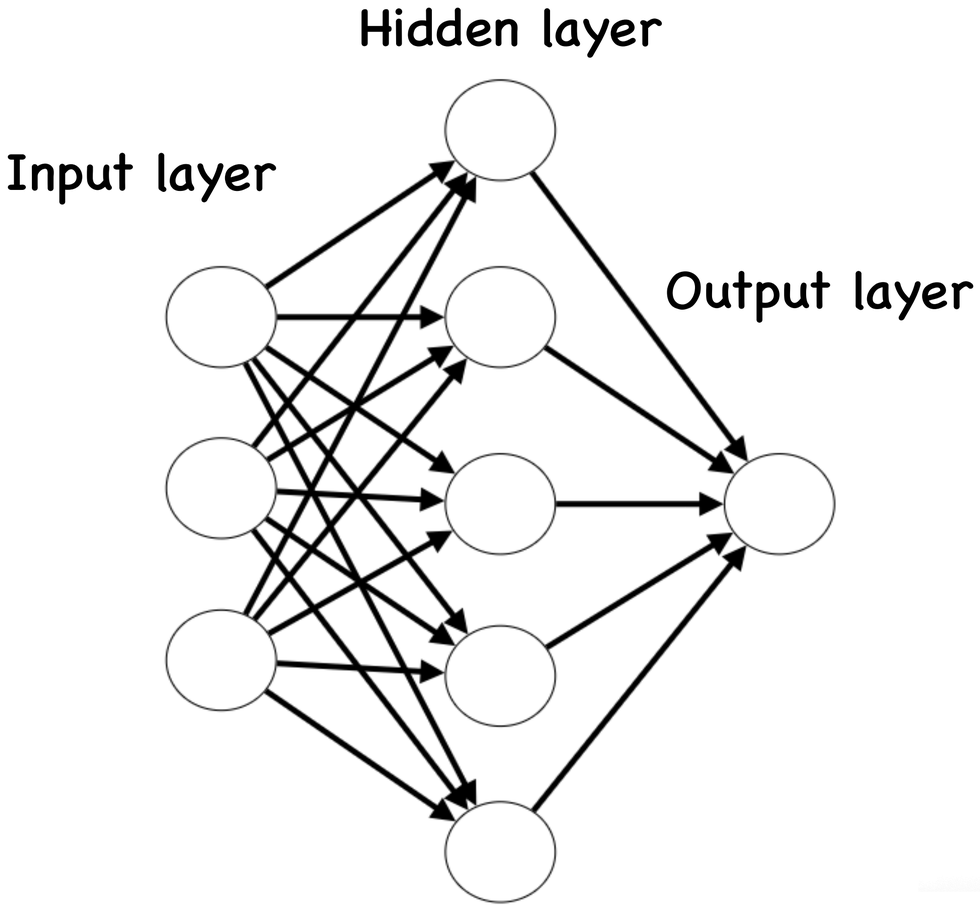
Neural Network



Universal approximation theorem

There exists a **neural network** with **one hidden** layer with a **finite** number of nodes and a non-polynomial activation function approximate any reasonable function to arbitrary precision

Neural Network

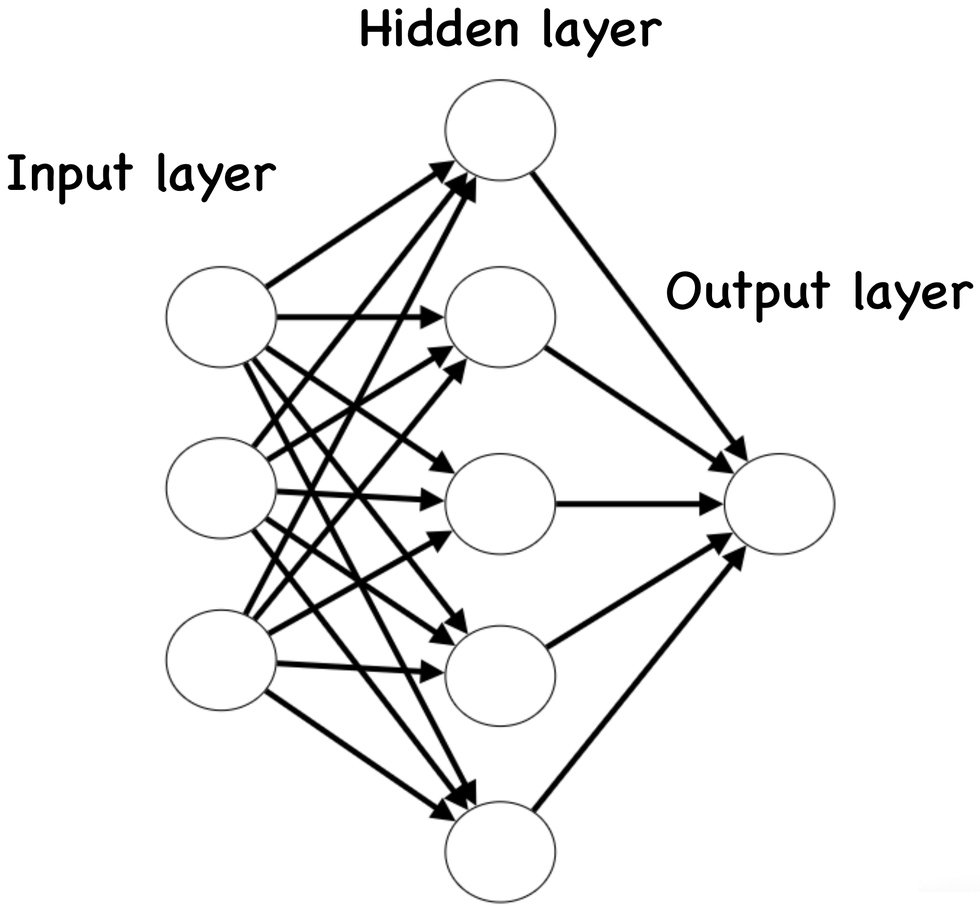


Universal approximation theorem

There exists a neural network with one hidden layer with a finite number of nodes and a non-polynomial activation function approximate any reasonable function to arbitrary precision

- Large enough network can approximate any function
- Only guarantees existence, not being able to find it
- In practice: one huge layer not great for training

Neural Network

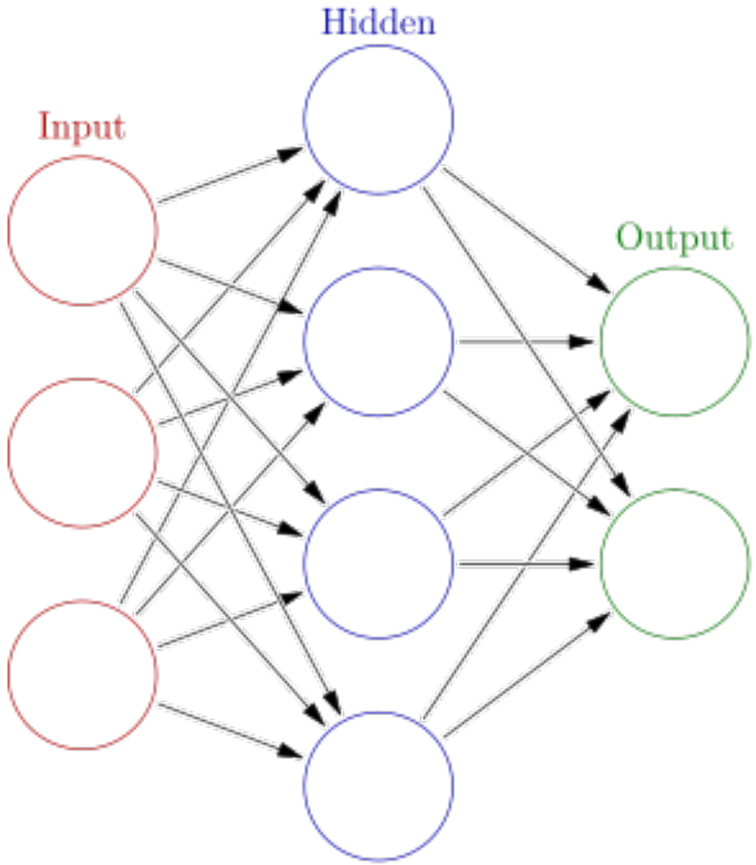


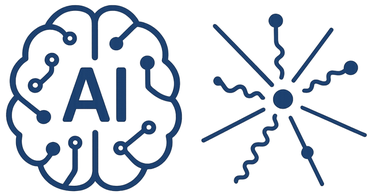
Universal approximation theorem

There exists a neural network with one hidden layer with a finite number of nodes and a non-polynomial activation function approximate any reasonable function to arbitrary precision

- Large enough network can approximate any function
- Only guarantees existence, not being able to find it
- In practice: one huge layer not great for training

Neural Network





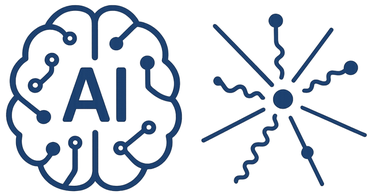
Softmax

For a vector of logits $z = [z_1, z_2, \ldots, z_C]$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Logits are unnormalized scores

C is the number of output (classes)



Softmax

For a vector of logits $z = [z_1, z_2, \ldots, z_C]$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Logits are unnormalized scores

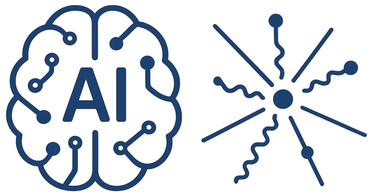
C is the number of output (classes)

z_i can be any real number (positive or negative)

e^{z_i} makes all values positive and amplifies larger ones.

outputs are between 0 and 1, and together they sum to 1

each output i can be interpreted as the probability that the input belongs to class i



Cross-Entropy Loss

What does this function called in physics ?

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

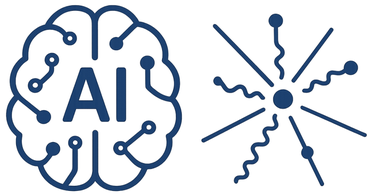
Cross-Entropy Loss

What does this function called in physics ?

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

The Boltzmann (Gibbs) distribution

$$p_i = \frac{1}{Q} \exp\left(-\frac{\varepsilon_i}{k_\mathrm{B}T}\right) = \frac{\exp\left(-\frac{\varepsilon_i}{k_\mathrm{B}T}\right)}{\sum_{j=1}^{M} \exp\left(-\frac{\varepsilon_j}{k_\mathrm{B}T}\right)}$$



Cross-Entropy Loss

What does this function called in physics ?

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

The Boltzmann (Gibbs) distribution

$$p_i = \frac{1}{Q} \exp\left(-\frac{\varepsilon_i}{k_\text{B}T}\right) = \frac{\exp\left(-\frac{\varepsilon_i}{k_\text{B}T}\right)}{\sum_{j=1}^{M} \exp\left(-\frac{\varepsilon_j}{k_\text{B}T}\right)}$$

Ex3 : Prove that provided the Boltzmann distribution the loss function is given by

$$L = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{c} y_{n,i} \log(\hat{y}_{n,i})$$